Autonomous Unitree Go2 Control with ArUco Markers and OpenCV

Full Course Module Plan

A course framework for teaching visual command signs, robot actions, scanning, patrol behavior, and built-in obstacle avoidance.

Course Concept

In this course, students learn how to use computer vision to control a Unitree Go2 robot dog with printed ArUco markers. The markers act as visual command signs that the robot can read through its camera.

Go2 camera sees marker
OpenCV identifies marker ID
Python connects that ID to a robot behavior
Go2 performs the behavior

The final project is not just a marker scanner. It is a visual command system for a mobile robot. The robot can patrol a space, scan for signs, react to different marker IDs, and use built-in obstacle avoidance to reduce collision risk.

Marker ID	Starter Behavior	Advanced Possibility
0	Stop	Emergency stop marker
1	Stand	Stand at demo zone
2	Sit	Rest or wait zone
3	Move forward	Continue path marker
4	Turn / scan	Turn-around or search marker
5	Not used at first	Pounce / jump-style action
6	Not used at first	Wave
7	Not used at first	Shake hands
8	Not used at first	Heart hands / performance gesture

Functional Use Cases

Security or Patrol Robot

The Go2 patrols a hallway, lab, or room. Markers act as signs that tell the robot what to do at different locations.

- Stop near a restricted area
- Sit at a checkpoint
- Turn around near a corner marker

Classroom Robotics Demo

Students print markers and immediately see the robot respond. This makes computer vision physical and easy to understand.

- Hold marker 1 to make the robot stand
- Hold marker 2 to make the robot sit
- Hold marker 0 to stop movement

Lab or Warehouse Navigation Signs

Markers can be placed around an environment as low-cost robot-readable labels.

- Loading zone marker
- Inspection area marker
- Turn-around marker

Human-Robot Interaction

A person can show a marker instead of using a joystick or app. The marker becomes a simple visual language.

- Show a gesture marker
- Show a stop marker
- Show a command marker during a demo

Full Course Module Plan

00 - Course Overview

Purpose: Explain the big idea, final project, use cases, and learning goals.

Main topics:

- Visual command signs for the Go2
- What students will build
- Why the project matters
- Use cases such as patrol, classroom demo, lab signs, and human-robot interaction

01 - What Is an ArUco Marker?

Purpose: Introduce ArUco markers as printed visual signs that a robot camera can detect.

Main topics:

- Black-and-white square marker
- Unique marker ID
- Black border and inner pattern
- Why markers are useful in robotics

02 - ArUco Dictionaries and Marker IDs

Purpose: Explain how marker dictionaries work and why the generator and detector must match.

Main topics:

- DICT_4X4_50
- 4x4 internal binary grid
- 50 possible IDs
- IDs become robot commands

03 - Creating and Printing ArUco Markers

Purpose: Show how to generate marker PNGs, print them, and test them reliably.

Main topics:

- Generate IDs 0-8
- Keep full black border
- Use good lighting
- Print large enough for room testing

04 - What Is OpenCV?

Purpose: Explain OpenCV as the computer vision library that processes Go2 camera frames.

Main topics:

- Computer vision basics
- Frame processing
- ArUco detector
- Marker ID returned to Python

05 - Go2 Camera and WebRTC Setup

Purpose: Teach how Python connects to the Go2 camera and robot data channel.

Main topics:

- Robot IP 192.168.12.1
- Virtual environment
- WebRTC connection
- Camera callback method
- Connection troubleshooting

06 - First ArUco Scanner

Purpose: Build the first scanner that displays the camera feed and prints marker IDs.

Main topics:

- Open camera window
- Detect markers
- Draw boxes
- Print IDs
- Test one marker at a time

07 - Understanding the Scanner Code

Purpose: Walk through the scanner code line by line so students know what each part does.

Main topics:

- Imports
- Dictionary setup
- Detector setup
- Camera callback
- Frame conversion
- Detection loop

08 - Triggering Go2 Actions from Markers

Purpose: Map marker IDs to built-in Go2 behaviors.

Main topics:

- Marker 0 stop
- Marker 1 stand
- Marker 2 sit
- Marker 3 forward
- Marker 4 turn/search

09 - Advanced Built-In Go2 Actions

Purpose: Add more expressive Go2 behaviors after the basic actions are stable.

Main topics:

- Pounce / jump-style action
- Wave
- Shake hands
- Heart hands
- Safety considerations

10 - Marker Confirmation and Cooldowns

Purpose: Make detection more reliable so commands do not repeat too quickly.

Main topics:

- Confirmation count
- Cooldown timer
- Last-seen tracking
- Stop before action

11 - 360-Degree Marker Scanning

Purpose: Make the robot rotate in place to search for markers around the room.

Main topics:

- Rotate slowly
- Scan camera feed
- Stop when marker appears
- Confirm marker
- Run action

12 - Slight Movement While Scanning

Purpose: Add slow movement while the robot searches, bridging toward patrol.

Main topics:

- Slow forward movement
- Slight turn
- Stop immediately on marker
- Safe testing space

13 - Autonomous Patrol Behavior

Purpose: Build the patrol state machine used in the working demo.

Main topics:

- FORWARD state
- SCAN state
- TURN state
- Return to patrol after marker action

14 - Built-In Go2 Obstacle Avoidance

Purpose: Enable and verify the Go2 built-in obstacle avoidance system.

Main topics:

- Query obstacle avoidance state
- Enable obstacle avoidance
- Confirm enabled=True
- Explain Python vs robot safety roles

15 - Wall and Corner Recovery

Purpose: Add a failsafe for blocked forward movement near walls or corners.

Main topics:

- Visual motion check
- Detect low camera motion while moving forward
- Back up
- Turn around
- Resume patrol

16 - Final Project Demo

Purpose: Students combine all modules into an ArUco patrol robot dog demonstration.

Main topics:

- Room setup with markers
- Robot patrols
- Robot reacts to signs
- Students customize marker behaviors

Recommended Teaching Path

1. Explain ArUco markers and dictionaries
2. Generate and print markers
3. Connect to Go2 camera through WebRTC
4. Detect marker IDs with OpenCV
5. Trigger safe actions: stop, stand, sit
6. Add marker confirmation and cooldowns
7. Add 360-degree scanning
8. Add slow movement while scanning
9. Add autonomous patrol
10. Enable built-in obstacle avoidance
11. Add blocked-wall recovery
12. Build final room patrol demo

Safety and Testing Notes

- Start with stop, stand, and sit before testing movement.
- Keep the robot on the floor with clear open space around it.
- Use slow movement settings for the beginner version.
- Keep one person ready to stop the script with q or CTRL+C.
- Close the Unitree app, browser UI, and old Python scripts before connecting through WebRTC.
- Treat jump, pounce, wave, shake hands, and heart hands as advanced actions that require extra space.

Current Working Status from Development

- Go2 camera stream works through unitree_webrtc_connect and WebRTC.
- OpenCV ArUco detection works with DICT_4X4_50.
- Markers 0-4 were detected successfully.
- Marker IDs can trigger real Go2 sport commands such as StopMove, StandUp, and Sit.
- Built-in Go2 obstacle avoidance was queried and confirmed enabled=True with code=0.
- Autonomous patrol script was created as go2_aruco_autonomous.py.
- The camera callback method conn.video.switchVideoChannel(True) and conn.video.add_track_callback(...) is the working method for this setup.
- The current patrol system can walk, scan, turn, stop on marker detection, trigger actions, and attempt blocked-wall recovery.

Useful Commands Appendix

Open the project folder in VS Code:

```
cd ~/projects/go2_aruco_opencv
code .
```

Activate the virtual environment:

```
cd ~/projects/go2_aruco_opencv
source venv/bin/activate
```

Run the current autonomous ArUco patrol script:

```
python go2_aruco_autonomous.py
```

Open the course documentation folder:

```
code course_docs
```